Chapter 8 | Data Structures & Algorithms

8.1 Classifying Data Structures

Linear (one predecessor, one successor)	Non-linear (branching)
Array, stack, queue, linked list — also circular queue and deque. All three of stack, queue and linked list **can be implemented on top of an ARRAY**	**Tree and graph** — the pair to remember; asked six times across the bands in every combination

(1) **Basic operations** on a data structure: insertion, deletion, traversal, searching, sorting, merging — **encryption is NOT one** (it is a security transformation). For **efficient searching AND insertion AND deletion together, the HASH TABLE** is the structure (average O(1) each) — the classic **time-space trade-off**: extra table space buys speed.

(2) **Dictionary<TKey, TValue>** is the structure holding **values indexed by a key**. A **stack** is the structure used when **processing must be postponed** until other conditions are fulfilled (LIFO deferral). A **tree can never contain a cycle** (n nodes, n − 1 edges); graphs, circular queues and circular lists all can.

Exam Points — 8.1

✦ Linear: array/stack/queue/linked list (buildable on arrays); non-linear: tree + graph. Encryption is not a data-structure operation.

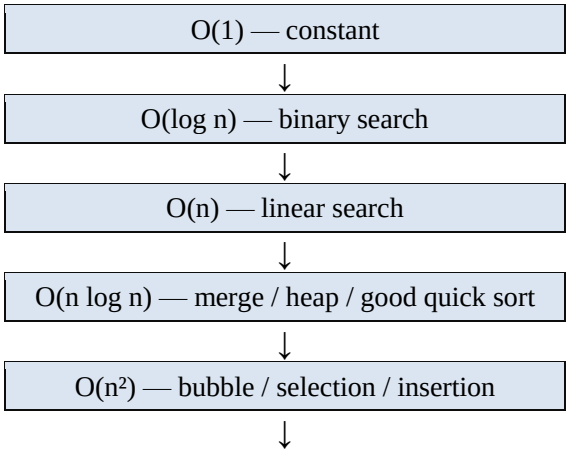
✦ Hash table = efficient search + insert + delete, the time-space trade-off. Dictionary = key-indexed values. Stack postpones processing. Trees cannot have cycles.

8.2 Complexity Analysis and Asymptotic Notation

Running time is analysed in three cases — **best, average and worst; 'test case' does not exist**. Three notations bound it:

Notation	Bound
Big-O	**Upper bound** — grows no faster than (asked twice)
Ω (Omega)	**Lower bound** constraint
Θ (Theta)	**Asymptotically tight upper AND lower bound** together

Only the fastest-growing term survives — constants and lower-order terms are dropped: $f(x) = 4x^2 - 5x + 3$ is simply **$O(x^2)$**. The growth ladder (asked three times in different subsets):



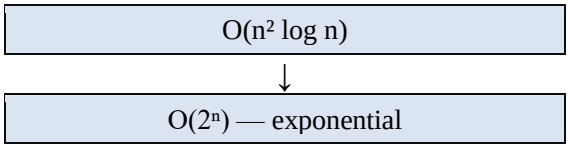


Fig. 8.1 — The growth ladder, slowest-growing first. Average-case ordering of the asked four: binary search < linear search < quick sort < selection sort.

(1) **Worked loop analyses**: the repeated-squaring exponentiate() halves m each time → **Θ(log n)**; abc() with outer i = n/2…n and inner j doubling → **O(n log n)**; the doubling loop j = j*2 up to n makes **$\log_2 n + 1$ comparisons**.

Recurrences to recognise:

$$T(n) = T(n/2) + 1 \Rightarrow \text{binary search (divide \& conquer)}, O(\log n)$$
$$T(n) = T(n-1) + Cn \Rightarrow \text{quicksort's WORST case}, O(n^2)$$

(2) **Tractable = solvable in polynomial time** (class P); intractable = exponential; the **halting problem is NOT COMPUTABLE at all** (undecidable — no algorithm exists). **Conditional asymptotic notation** is used when **behaviour varies with input characteristics** (e.g. n a power of 2). **Strassen's matrix multiplication runs in $O(n^{2.81})$** (7 multiplications, exponent $\log_2 7$); a single-tape Turing machine simulates n moves of a multi-tape machine in **$O(n^2)$**.

(3) Choosing an efficient data structure happens in the **algorithm DESIGN phase**. Operation-count reasoning: N insertions but only √log N other operations → pick the **unsorted array** (O(1) insert). A **dynamic array append is O(1) normally but O(n) when it must grow — 'either O(1) or O(n)'**, amortised O(1).

Exam Points — 8.2

- ✦ Cases: best/average/worst (no test case). O = upper, Ω = lower, Θ = tight (×2). $4x^2-5x+3 \rightarrow O(x^2)$. Ladder: $1 < \log n < n < n \log n < n^2 < n^2 \log n < 2^n$.
- ✦ exponentiate Θ(log n); abc O(n log n); doubling loop $\log_2 n + 1$. T(n)=T(n/2)+1 = divide & conquer; T(n)=T(n−1)+Cn = quicksort worst.
- ✦ Tractable = polynomial; halting problem not computable; conditional notation = input-dependent behaviour; Strassen $O(n^{2.81})$; 1-tape simulation $O(n^2)$; structure choice = design phase; dynamic append O(1)/O(n).

8.3 Arrays and Address Calculation

(1) An array is stored **contiguously in a single memory block** — which is why it enjoys **spatial locality** (touching one element caches its neighbours). **height[0] is the first element** (indices 0 to n − 1); the largest index is the **upper bound**; **length = j − i + 1** for first index i, last j; E[−4 : 1, 6 : 10] has 6 × 5 = **30 elements**. In a 2D character array, the **first dimension = number of names, second = maximum length of each**.

(2) **Insertion at position i shifts elements i … n up to become i+1 … n+1**; worked: inserting 45 at index 4 of [10, 20, 30, 40, 50, 60] → **[10, 20, 30, 40, 45, 50, 60]**. **Deleting the first element needs n − 1 shifts — O(n)**; converting a linked list to an array (one traversal) is **O(n)**.

Address calculation — the formulas the papers compute with (B = base, w = element width, C = columns):

$$\text{Row-major 2D: } LOC(A[i][j]) = B + w \times (i \times C + j)$$
$$\text{Row-major 3D: } LOC(A[i][j][k]) = B + w \times ((i \times d_2 + j) \times d_3 + k)$$

Worked: B[4][5], base 5000, w = 2: LOC(B[2][3]) = 5000 + 2(2×5 + 3) = **5026**. C[3][4][5], base 3000, w = 1: LOC(C[2][3][4]) = 3000 + (2×4 + 3)×5 + 4 = **3059**. (Column-major swaps the roles: B + w(j × rows + i).)

(3) **Tri-diagonal sparse matrix** (non-zeros only where $|i - j| \le 1$): stored row-wise in a 1-D array, element (i, j) sits at index **2i + j − 2**, and only **3n − 2 cells** are needed.

Exam Points — 8.3

✦ Contiguous single block → spatial locality; first element = [0]; upper bound = largest index; length = j−i+1; E[−4:1, 6:10] = 30; 2D char array = names × max length.

✦ Insert at i renumbers to i+1…n+1 (worked 45); delete first = n−1 shifts, O(n). Row-major workeds 5026 and 3059; tri-diagonal index 2i+j−2.

8.4 Linked Lists

(1) A singly linked node = **data value + address of the NEXT node**; the head is the first node and the last node's pointer is NULL. The list is **linear**, its size **grows and shrinks dynamically with memory allocated per node** (statements P and R) — but its elements are **only sequentially accessible, never randomly** (the incorrect claim, asked three times): reaching an index costs O(n), and that is the list's tested disadvantage.

(2) **Advantages over arrays**: efficient **insertion and deletion at any position** (pointer changes, no shifting); **insertion at the head is O(1)** (asked twice) — the tail is O(n) without a tail pointer. **Searching is O(n)**; done recursively it is **O(n) time AND O(n) space** (stack frames).

(3) **Doubly linked list**: next + previous pointers → **traversal in BOTH directions**; inserting between two nodes changes **FOUR pointers** (new node's two, both neighbours' one each — a singly list needs two). A **deque built on a SINGLY linked list deletes at the rear in O(N)** (must walk to the predecessor); a doubly linked list makes it O(1).

(4) **Circular list**: last node points to the first — the structure for **round-robin CPU allocation**. Deleting the first node of a circular list with START and END: **START = START→NEXT; END→NEXT = START** (advance the head, then repair the tail's link).

(5) **Sorting and searching on lists**: **merge sort is the best sort for a linked list** (sequential access suffices); **binary search is NOT suitable for linked lists** (no O(1) access to the middle — asked three times). Visiting every node once = **linear traversal** (pre/in/post-order belong to trees). The list-vs-array truth set: array size fixed, list dynamic; array insertion/deletion expensive; **'random and direct access in a list' is the false statement**.

Exam Points — 8.4

✦ Node = data + next; sequential access only (×3); dynamic size with per-node allocation; search O(n); recursive search O(n)/O(n).

✦ Head insert O(1) (×2); insert/delete anywhere = the advantage; doubly = both directions, 4 pointer changes; SLL-deque rear delete O(N); circular → round-robin; circular head delete = advance START + repair END.

✦ Merge sort best for lists; binary search unsuitable (×3); linear traversal; list never gives random access.

8.5 Stacks and Expression Notation

A **stack is LIFO** (asked five times): push and pop at ONE end, the top. **Popping an empty stack = UNDERFLOW** (pushing a full one = overflow). Worked: stack L, M, N, O, P, Q (top L) after three removals → **O on top**. The five-item shuffle (P…T pushed, four popped into a queue, two dequeued and pushed back) → next pop = **S**.

(1) **Applications** (asked across seven papers): **recursion / function calls** — each call pushes an activation record with parameters, locals and the return address (×4); compiler **syntax analysis**; **tracking local variables**; expression conversion and evaluation; undo; backtracking; **symbol balancing**. NOT a stack job: asynchronous data transfer between processes (that is a queue/pipe).

Notation	Facts
Infix	Operators between operands; needs precedence and brackets
Prefix (Polish)	Operator FIRST. A + B*C → + A * B C ; and (A*B + C)/(E − F*G) → / + * A B C − E * F G. Evaluated with a **stack**, scanning right to left

Postfix (Reverse Polish)	Operator LAST — no brackets, no precedence; abc*+de−+ is the postfix string in the identification question. Evaluated with a **stack**, left to right; conversion infix→prefix/postfix also uses a **stack**

Worked: Prefix * + a b − c d → postfix ab+ cd− * . Postfix M N O + * P / Q R S T / + * − → infix (((M*(N+O))/P) − (Q*(R+(S/T)))). Evaluating 3 2 + 7 * 6 5 + 4 * −: just before the second +, the stack holds **5 on top, 35 at the bottom**.

Exam Points — 8.5

- ✦ Stack = LIFO, one end; empty pop = underflow; pop-3 → O; shuffle → S. Applications: recursion/calls (×4), syntax analysis, locals, expressions, balancing; not async transfer.
- ✦ +A*BC; /+*ABC−E*FG; ab+cd−*; the parenthesised M-N-O infix; postfix = abc*+de−+; stack for all conversions and evaluations; postfix snapshot 5/35.

8.6 Queues — Simple, Circular, Deque, Priority

A **queue is FIFO** and **linear**: **insertion only at the REAR, deletion only at the FRONT** (asked five times). Elements leave in arrival order — A, B, C, D come out **A B C D**; reversing a 4-element queue to d, c, b, a needs **3 deletions + 3 additions**. Real-world models: **toll plaza simulation**, message queues (messages held **in a queue structure**), printer sharing, asynchronous data transfer. **Symbol balancing is NOT a queue application** (stack). Check **UNDERFLOW before deletion** (overflow before insertion).

(1) **Linear queue overflow: rear = MAX_SIZE − 1** — even when the front has free slots (false overflow). The cure is the **circular queue (= RING buffer)**, whose purpose is to **avoid wastage of memory**. Conditions: **empty → front = rear = −1**; one element → **front = rear ≠ null**; full → (rear + 1) mod MAX = front.

(2) Linked-queue insertion: **REAR→next = PTR; REAR = PTR** (never touch FRONT). A **deque** allows insertion/deletion at **either end but not the middle**. A **priority queue is best implemented with a BINARY HEAP** (asked twice): insert and extract both O(log n).

Note: Three OS strays parked here (full treatment in Chapter 3): the **blocked queue holds processes waiting for an event/resource such as I/O**; per-thread state lives in the **Thread Control Block**; and **burst time = CPU execution time required, not the time to start** (the false definition).

Exam Points — 8.6

- ✦ FIFO; rear in, front out (×5); ABCD unchanged; reversal = 3 del + 3 add; underflow before dequeue; toll plaza and message queues; balancing is not a queue job.
- ✦ Linear overflow rear = MAX−1; circular = ring buffer, saves memory; empty = both −1; single element front = rear; REAR→next=PTR, REAR=PTR; deque = both ends; priority queue = binary heap (×2).

8.7 Trees — Terminology, Traversals, BST

(1) **Terminology**: the number of subtrees (children) of a node = its **DEGREE**; a node of **degree 0 = leaf/terminal node**; **level = distance from the root** (root at level 0); height = longest root-to-leaf path. A tree is **non-linear** and **acyclic** (n nodes ⇒ n − 1 edges).

Counting identities — each one a direct question:

$$Height\ n:\ max\ nodes\ =\ 2^{\,n+1}\ -\ 1\,,\ \ min\ nodes\ =\ n\ +\ 1$$

$$Complete\ tree\ of\ level\ 5:\ 2^{\,6}\ -\ 1\ =\ 63 \qquad Full\ tree, n\ internal:\ 2n\ +\ 1\ nodes$$

$$n\ leaves\ \Rightarrow\ n\ -\ 1\ degree - 2\ nodes \qquad n\ nodes\ \Rightarrow\ n\ +\ 1\ NULL\ pointers \qquad 4\ keys\ \Rightarrow\ 14\ BSTs\ (Catalan)$$

(2) **Traversals**: pre-order = NLR, in-order = LNR, **post-order = LRN** (Left, Right, Node — direct question). Invented 'new-orders' just permute the letters: right-left-node on an alphabet tree → **K, J, I … A**; right-node-left → **Z, V, Y,**

X, W, P, S, U, T, Q, R. DFS on trees/graphs is implemented with a stack (BFS with a queue). A **threaded binary tree** turns the n + 1 null pointers into threads to the **inorder predecessor or successor** — making **finding the predecessor/successor faster** and traversal possible without recursion or stack.

(3) **BST property: left < node < right** — the right subtree holds the GREATER values; the tested statement pair reversing this is 'neither'. **In-order traversal of a BST is ascending sorted order** (asked twice; the 70, 50, 10… insertion worked prints 10…20…100). BST operations = search, insert, delete — **sorting is not an operation** (sorted output is the by-product of in-order). Worst-case search = **O(n)** (degenerate chain); height ranges from **O(log₂N) best to N − 1 worst**.

(4) **Deletion rules**: a leaf → replace the parent's link with NULL (true); a two-child node → replace with the **INORDER successor — not preorder** (the false statement). Worked deletions: removing 14 → pre-order **16, 10, 7, 13, 11, 20, 18, 25, 22**.

Worked: Traversal reconstructions: preorder 38,14,8,23,18,20,56,45,82,70 → postorder **8, 20, 18, 23, 14, 45, 70, 82, 56, 38**. Inorder EICFB GDJHK + postorder …B → preorder **B, C, E, I, F, D, G, H, J, K**. Inorder QSPTRU + preorder PQSRTU → postorder **S, Q, T, U, R, P**. BST preorder Y X A B Z C → postorder **A, B, X, C, Z, Y**. Valid BST preorder of 1–8: **53124768**. Inserting 71, 56, 48, 96, 76, 82, 98 → deepest node **82**.

(5) Application strays kept at the same level: a parse tree built **root-first in preorder = TOP-DOWN parsing**; game trees decide win/lose with the **MIN/MAX algorithm**; **pruning a decision tree always REDUCES its size**; **McCabe's cyclomatic complexity V(G) = E − N + 2** (software-engineering metric, its formula asked here).

Exam Points — 8.7

- ✦ Degree = subtree count; leaf = degree 0; level = distance from root; tree acyclic, non-linear. Bounds: $2^{n+1}-1$ / n+1; level-5 = 63; full = 2n+1; leaves−1 = degree-2 nodes; n+1 nulls; Catalan 14.
- ✦ Post-order = LRN; BST in-order = ascending (×2); right subtree = greater; delete leaf → null link, two children → inorder successor; worst search O(n); height $\log_2 N$ … N−1; all six reconstruction workeds; DFS = stack; threads = fast predecessor/successor.
- ✦ Top-down parsing = root-first preorder; game tree = minimax; pruning shrinks the tree; V(G) = E − N + 2.

8.8 Balanced Trees and Heaps

Structure	Asked facts
AVL tree	**Balance factor ∈ {−1, 0, +1}**; the subtree heights of every node differ by **at most 1**; **unbalanced (rotation needed) when any node reaches ±2**; worked deletion left BF(14) = **−2**. Rotations: **LL and RR = single; LR and RL = double**. AVL beats a plain BST in the worst case for **search AND insert AND delete** (all three)
Red-black tree	**A new root node is coloured BLACK; a non-root insertion is coloured RED** (more than one rule holds); height ≤ **2 $\log_2(n + 1)$**
B-tree	**NOT a binary tree** (many keys per node) — the balanced non-binary answer; the **most widely used EXTERNAL-memory structure** (node = disk block); root has 2…M children; a node is full with M − 1 keys; **all leaves at the SAME level** ('not at the same level' is the false statement)
Heap	Complete binary tree with the heap order; array child/parent arithmetic (1-based: parent = ⌊ i/2⌋). **Max-heap insertion sifts the new key up**: 400-heap + 350 → **400, 350, 200, 100, 300, 160, 170, 80, 40, 150**; 23-heap + 20 → **23, 20, 14, 17, 13, 10, 1, 5, 6**. Heapifying the 289…300 array needs **3 interchanges**. The heap implements the **priority queue**

Exam Points — 8.8

✦ AVL: BF ∈ {−1,0,1}, differ ≤ 1, rotate at ±2 (worked −2); LL/RR single, LR/RL double; better worst case for all three operations.

✦ Red-black: root black / non-root red; height ≤ $2\log_2(n+1)$. B-tree: balanced but not binary; external memory; leaves level-equal. Heap insert workeds (both sequences); heapify = 3 swaps; heap → priority queue.

8.9 Graphs — Degrees, Traversal, Spanning Trees, NP

(1) Degrees: a degree-1 vertex is **PENDANT**; a path graph on n vertices has **n − 2 vertices of degree 2**. The **handshaking lemma** (both tested statements true — odd-degree vertices are even in number, and the degree sum is even):

$$\Sigma\, deg(v) = 2 \times |E| \qquad max\ edges\ (simple\ graph) = n(n-1)/2 \rightarrow n = 8 \Rightarrow 28$$

(2) Representation and traversal: the vertex-connection matrix = **ADJACENCY matrix**. **BFS uses a QUEUE, DFS a STACK** (the ordered pair (Queue, Stack)); **DFS takes less memory** (asked twice — stores one path, not a frontier). **Articulation points** (removal disconnects) in the worked graph: **2, 3 and 5**. Worked weighted-digraph shortest distance V1→V5 = **120**.

(3) Spanning trees: n vertices ⇒ exactly **n − 1 edges** (asked twice; 'n/2 edges' is the invalid MST property — an MST has n vertices, n − 1 edges, and is connected). **Kruskal**: greedy, picks edges in **increasing weight order**, rejects cycles — **all three statements true**; Prim grows from a vertex (also greedy). **Dijkstra uses a MIN-priority queue**; its AI twin, **uniform-cost search, expands the node with the LOWEST path cost**. **Bellman-Ford on a complete graph costs $\Theta(n^3)$** (V−1 passes × ~n^2 edges; handles negative weights).

(4) NP-completeness: TSP, Boolean satisfiability, 3-CNF-SAT, clique, Hamiltonian cycle and vertex cover are NP-complete. **The SHORTEST PATH problem is NOT** (asked twice — Dijkstra solves it in polynomial time), and **DFS is not** (it is an algorithm, O(V + E), not a problem).

Exam Points — 8.9

✦ Pendant = degree 1; path graph → n−2 degree-2 vertices; Σdeg = 2|E| (odd-degree count even); max edges n(n−1)/2 = 28; adjacency matrix; articulation 2,3,5; shortest 120.

✦ BFS queue / DFS stack; DFS lighter on memory (×2). Spanning tree = n−1 edges (×2); Kruskal all-true; Dijkstra min-priority queue; uniform-cost = lowest path cost; Bellman-Ford $\Theta(n^3)$; shortest path & DFS not NP-complete.

8.10 Searching and Hashing

(1) Linear (sequential) search: O(n) worst case; **average comparisons = (n + 1)/2**; needs no sorting. A **self-organising list** (move-to-front) exploits **locality of reference**. Picking the best solution from a set of candidates = an **OPTIMISATION problem**.

(2) Binary search: needs a **SORTED array + direct access to the middle** (statements I and II; **not implementable on a linked list** — asked three times, and it **does not build a BST** — the invalid statement). Worst AND average case **O(log n)**; the false claim is best case O(log n) — the best case is one comparison. **Unsuccessful when End < Beg**. Its recurrence is the divide-and-conquer T(n) = T(n/2) + O(1):

$$max\ comparisons = \lceil log_2 n \rceil:\ 32 \rightarrow 5,\ 12{,}000 \rightarrow 14,\ 10{,}00{,}000 \rightarrow 20$$

(3) Hashing = deriving the STORAGE ADDRESS from the record key — one computation instead of a search. **Hash functions**: division, mid-square, folding — **the probe method is NOT a hash function**, and **folding is not a collision-resolution technique** (the two directions of the same trap). Division method:

$$h(k) = k\ mod\ m:\ m = 57 \Rightarrow h(177) = 6,\qquad h(197) = 26$$

(4) Collision resolution: **separate chaining stores collided keys in LINKED LISTS** hung off the slot; **linear probing takes the NEXT available slot** (suffers primary clustering); quadratic probing, double hashing and rehashing are

also valid — **CLUSTERING itself is a problem, not a technique**. Worked linear probing: keys 22, 28, 23, 12, 13 into a length-9 table, h(k) = k mod 9 → 13 collides at 4, probes past 5 → **index 6**.

Exam Points — 8.10

- ✦ Linear search: O(n), average (n+1)/2, unsorted OK; self-organising list = locality of reference; best-of-set = optimisation.
- ✦ Binary search: sorted + middle access, no linked lists (×3), builds no BST, worst/average O(log n) (best = 1), End < Beg = failure; 5 / 14 / 20 comparison workeds.
- ✦ Hashing: key → address; functions = division/mid-square/folding (probe is not); h(k) = k mod 57 → 6, 26. Resolution = chaining (linked lists), linear/quadratic probing, double hashing, rehashing; folding and clustering are not; probing worked → index 6.

8.11 Sorting Algorithms

Algorithm	Complexity and asked facts
Bubble	Worst O(n²); **worst-case comparisons = n(n−1)/2 → 15 elements = 105**; **swaps = number of inversions** → "SORTED" = **11**, and 8,22,7,9,31 = **3**; pass worked: 37,54,21,85,68,12,9,57 after pass 2 → **21, 37, 54, 12, 9, 57, 68, 85**
Selection	**Best = worst = O(n²)** (always scans for the minimum); **maximum swaps n − 1** — its advantage is the tiny swap count (the BPSC key words it 'only n swaps'); pass worked: H,V,A,T,L,N,K after pass 3 → **A, H, K, T, L, N, V**
Insertion	Stable; iterative. (One 2025 key marks it 'worst on nearly-sorted input' — the book's own explanation flags this as inconsistent, since insertion sort is nearly O(n) there; verify against the paper)
Merge	**Divide and conquer**; **O(n log n) in every case including average**; **STABLE** (the stable one among selection/quick/heap/merge); recursive; best for linked lists
Quick	**Divide and conquer** (asked twice); average O(n log n); **worst Θ(n²)** (sorted input, extreme pivot; recurrence T(n−1) + Cn); **with an O(n) MEDIAN pivot → O(n log n) guaranteed**; recursive
Heap	**O(n log n) worst case guaranteed**; in-place; NOT stable
Radix	Non-comparison, digit by digit; pass-2 worked (tens digit): 329,457,839,436,720,355,657 → **720, 329, 436, 839, 355, 457, 657**
Binary tree sort	Insert all into a BST + inorder; **worst O(n²)** (degenerate chain)

(1) **Stability** = equal keys keep their input order (definition question). **Stable pair: bubble + insertion** (also merge); unstable: selection, quick, heap. **Recursive pair: quick + merge** (the divide-and-conquer sorts); bubble and insertion are iterative.

(2) Task matching: recursion → stack, scheduling → queue, sorting → binary tree (A-ii, B-iii, C-i). Merge sort and binary search together: **O(n $\log_2$n) and O($\log_2$n)**.

Exam Points — 8.11

- ✦ Bubble: n(n−1)/2 comparisons (105); swaps = inversions (11; 3); pass-2 worked. Selection: O(n²) both cases, ≤ n−1 swaps, pass-3 worked. Radix pass-2 worked.
- ✦ Merge: D&C, O(n log n) all cases, stable (×2), list-friendly. Quick: D&C (×2), worst Θ(n²) via T(n−1)+Cn, median pivot → O(n log n). Heap: guaranteed O(n log n). Tree sort worst O(n²). Stable pair bubble+insertion; recursive pair quick+merge.

8.12 Algorithm Design Techniques

Technique	Identity and asked facts

Divide & Conquer	**Recursively solves subproblems**, then combines (divide–conquer–combine). Members: binary search, quick sort, merge sort, Strassen. **All-pairs shortest path (Floyd-Warshall) is NOT D&C** — it is DP. D&C max-min finding is still **O(n)** (saves comparisons ≈ 3n/2, not the order)
Greedy	**Locally optimal choice, ONE input considered at a time** (the two-statement question: 'divides the stream in two' false, 'one at a time' true); **faster than DP but does NOT examine all feasible solutions** (the false claim); members: **Kruskal, Prim, fractional knapsack, Dijkstra**
Dynamic Programming	For problems with **OVERLAPPING subproblems** (+ optimal substructure); members: **0/1 knapsack, LCS — O(n²)** for two length-n sequences, Floyd-Warshall, Bellman-Ford
Backtracking	Systematic trial with undo — subset-sum, n-queens. Worked subset sum w = {2,7,8,9,15}, target 17 → **11100 (2+7+8), 10001 (2+15), 00110 (8+9)**

Exam Points — 8.12

✦ D&C = recursive subproblems (binary search/quick/merge/Strassen); Floyd-Warshall is DP; max-min stays O(n). Greedy = local optimum, one input at a time, never exhaustive; fractional knapsack greedy, 0/1 knapsack DP.

✦ DP = overlapping subproblems; LCS $O(n^2)$. Subset-sum worked: 11100, 10001, 00110.

Quick Revision — 35 points

1. Linear: array, stack, queue, linked list (all array-implementable); non-linear: tree, graph. Encryption is not a DS operation; hash table = fast search+insert+delete (time-space trade-off); stack postpones processing; trees are acyclic.
2. Cases best/average/worst (no 'test case'). O upper, Ω lower, Θ tight. Drop constants: $4x^2-5x+3 = O(x^2)$.
3. Ladder: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^2 \log n) < O(2^n)$. Average order: binary search < linear search < quick < selection.
4. exponentiate Θ(log n); abc() O(n log n); j*=2 loop = $\log_2 n+1$. T(n)=T(n/2)+1 → D&C/binary search; T(n)=T(n−1)+Cn → quicksort worst.
5. Tractable = polynomial; halting problem = not computable; Strassen O(n^2.81); 1-tape simulation $O(n^2)$; structure choice in the design phase; dynamic-array append O(1) or O(n).
6. Array: contiguous block, spatial locality, first element [0], length j−i+1, E[−4:1,6:10] = 30, upper bound = largest index.
7. Insert at i → i+1…n+1 (worked 45); delete first = n−1 shifts. Row-major LOC = B + w(iC+j): 5026; 3D: 3059. Tri-diagonal: 2i+j−2, 3n−2 cells.
8. Linked node = data + next; sequential access only; head insert O(1); search O(n) (recursive: O(n) space too); doubly = both directions, 4 pointer changes; SLL rear-delete O(N).
9. Circular list → round-robin; head delete: START=START→NEXT; END→NEXT=START. Merge sort best for lists; binary search impossible on lists.
10. Stack LIFO; empty pop = underflow; applications recursion/calls/syntax/locals/expressions/balancing; not async transfer. Pop-3 → O; shuffle → S.
11. Prefix +A*BC; /+*ABC−E*FG; prefix→postfix ab+cd−*; postfix eval snapshot 5 top / 35 bottom; postfix = operators last (abc*+de−+); stack does every conversion/evaluation.
12. Queue FIFO, rear-in front-out; ABCD unchanged; reversal 3+3; underflow check before dequeue; toll plaza = queue; balancing ≠ queue.
13. Linear queue overflow rear = MAX−1 (false overflow); circular = ring buffer, avoids memory waste; empty front=rear=−1; one element front=rear; enqueue REAR→next=PTR, REAR=PTR.
14. Deque = both ends, never middle. Priority queue = binary heap (insert/extract O(log n)).
15. Tree: degree = children; leaf = degree 0; level = distance from root; n−1 edges. Max/min nodes $2^{n+1}-1$ / n+1; level-5 complete = 63; full = 2n+1; leaves−1 degree-2 nodes; n+1 null pointers; Catalan(4) = 14.
16. Traversals NLR/LNR/LRN; BST inorder = ascending; right subtree greater; delete two-child node with the inorder successor; BST worst search O(n); height $\log_2 N$ … N−1.

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B

17. Reconstructions: pre→post (8,20,18,23,14,45,70,82,56,38); in+post→pre (B,C,E,I,F,D,G,H,J,K); in+pre→post (S,Q,T,U,R,P); BST pre→post (A,B,X,C,Z,Y); valid preorder 53124768; deepest insert 82; delete-14 preorder option (b).
18. Threaded tree: nulls → inorder predecessor/successor threads, faster successor finding. Top-down parsing = preorder root-first; minimax for game trees; pruning shrinks; V(G) = E−N+2.
19. AVL: BF ∈ {−1,0,1}; rotate at ±2; LL/RR single, LR/RL double; beats BST worst case in search+insert+delete.
20. Red-black: root black, else red; height ≤ $2\log_2(n+1)$. B-tree: balanced non-binary, external memory, leaves level-equal, root ≥2 children, full = M−1 keys.
21. Heap: sift-up insert (400…350 and 23…20 workeds); heapify worked = 3 swaps; heap implements the priority queue.
22. Graph: pendant = degree 1; path graph n−2 middle vertices; Σdeg = 2|E|; odd-degree count even; max edges n(n−1)/2 = 28 for n=8; adjacency matrix; articulation 2,3,5.
23. BFS = queue, DFS = stack; DFS uses less memory. Spanning tree n−1 edges; Kruskal greedy/increasing/no-cycles all true; Dijkstra min-priority queue; uniform-cost = lowest path cost; Bellman-Ford $\Theta(n^3)$.
24. NP-complete: TSP, SAT/3-CNF, clique, Hamiltonian, vertex cover. Not NP-complete: shortest path (Dijkstra), DFS (an algorithm).
25. Linear search O(n), average (n+1)/2; self-organising list = locality; optimisation = best of a set.
26. Binary search: sorted + middle access, never on lists, no BST built; worst/average O(log n), best 1; End<Beg = fail; 32→5, 12000→14, 10^6→20.
27. Hashing = key → address; functions division/mid-square/folding (probe not); k mod 57: 177→6, 197→26.
28. Collisions: chaining (linked lists), linear probing (next slot, primary clustering), quadratic, double hashing, rehashing; folding & clustering are not techniques; probing worked → index 6.
29. Bubble: n(n−1)/2 = 105 comparisons; swaps = inversions (11, 3); pass-2 worked. Selection: $O(n^2)$ always, ≤ n−1 swaps, pass-3 A,H,K,T,L,N,V.
30. Merge: D&C, O(n log n) every case, stable, list-friendly. Quick: D&C, worst $\Theta(n^2)$, median pivot fixes it. Heap: guaranteed O(n log n), unstable. Radix pass-2 worked; tree sort worst $O(n^2)$.
31. Stable pair = bubble + insertion (merge too); recursive pair = quick + merge. Merge/binary-search pair = $O(n \log_2 n)$, $O(\log_2 n)$.
32. D&C = divide, conquer (recursively), combine; Floyd-Warshall is DP; D&C max-min still O(n).
33. Greedy: one input at a time, local optimum, fast, not exhaustive; Kruskal/Prim/Dijkstra/fractional knapsack. DP: overlapping subproblems; 0/1 knapsack, LCS $O(n^2)$.
34. Subset-sum {2,7,8,9,15}→17: 11100, 10001, 00110 (backtracking).
35. Recursion → stack; scheduling → queue; sorting → binary tree (the matching question).

DSSSB – TGT COMPUTER SCIENCE (PRACTICE KING) BOOK

Call/Message: 7011121290 Available @ Amazon: https://amzn.in/d/07E04o9B